

CLAUDE CODE 101

Claude Code 101

學習筆記

9 堂課帶你從零認識 Claude Code
從安裝到進階工作流，從 CLAUDE.md 到 Hooks
一份筆記，完整掌握

資料來源：整理自 [Claude Youtube 官方頻道課程 Claude-Code-101](#)

目錄 Contents

- 01 What is Claude Code? 什麼是 Claude Code ?
- 02 Installing Claude Code 安裝 Claude Code
- 03 How Claude Code Works Claude Code 如何運作
- 04 Your First Prompt 你的第一個 Prompt
- 05 The CLAUDE.md File CLAUDE.md 設定檔
- 06 Explore → Plan → Code → Commit 探索→計畫→程式設計→提交
- 07 Context Management 情境視窗管理
- 08 MCP 整合外部工具
- 09 Hooks in Claude Code 確定性自動化

What is Claude Code?

什麼是 Claude Code？

本節概念：Agentic Tool · AI Agent · Context Window · Permission Mode

► [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

1. Claude Code 是 **Agentic 程式設計工具**，可直接存取你的檔案、終端機與程式碼庫，不需要複製貼上。
2. 與 Claude AI 的差異：Claude Code 能「自己進去做事」，而非只輸出文字讓你自己操作。
3. **AI Agent** = 能與環境互動、自主執行行動來達成目標的軟體。
4. **Context Window（情境視窗）** = Claude 的工作記憶，有容量上限，無法存放所有內容。
5. 預設行為：執行操作前會詢問許可；Claude Code 並非完美，可能誤解意圖或引入錯誤。

核心概念 CORE CONCEPTS

Agentic Loop 代理循環

接收 prompt → 收集情境 → 呼叫模型 → 執行行動 → 驗證結果 → 若未達標則循環重試。

Tools 工具

讓 Agent 能「做事」的能力模組，例如讀取檔案、搜尋網頁、執行建置程式化腳本等。

Context Window 情境視窗

Claude 的工作記憶，存放對話記錄、檔案內容、指令輸出等。容量有限，需策略性管理。

關鍵詞彙 VOCABULARY

英文	中文	說明
Agentic Coding Tool	代理式程式設計工具	能自主執行多步驟任務的 AI 工具
AI Agent	AI 代理	能與環境互動、執行行動來完成目標的軟體
Context Window	情境視窗	Claude 的工作記憶，存放對話、檔案等內容
Permission Mode	許可模式	控制 Claude 執行操作前是否需要詢問確認
Tools	工具	讓 Agent 執程式碼、搜尋等行動的功能模組
Codebase	程式碼庫	整個專案的程式碼集合

可用平台 PLATFORMS

平台	特點
Terminal（終端機）	功能最完整，功能更新最快
VS Code	與編輯器深度整合
JetBrains IDEs	與 IDE 整合
Claude Desktop	適合背景執行、同時處理多任務
Claude.ai/code（Web）	可在瀏覽器直接執行任務，不限 GitHub；支援桌面瀏覽器與 iOS app

● 延伸資源 RESOURCES

Claude Code 官方文件

功能說明、指令速查與常見問題的完整參考手冊，學習途中遇到不懂的概念可直接查閱

官方入門課程 Claude Code 101

Anthropic 官方推出的完整免費線上課程，內容比 YouTube 短片系列更深入，完成後可取得官方認證證書

Claude Code 產品介紹頁

了解 Claude Code 的功能定位、適用場景與訂閱方案，適合剛接觸的學習者瞭解全貌

我的筆記 MY NOTES

Installing Claude Code

安裝 Claude Code

本節概念：Install · Platform Setup · Auto Update

▶ [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

1. **Mac / Linux / WSL**：用 `curl` 安裝（支援自動更新）；Homebrew 可用但**不自動更新**。
2. **Windows**：PowerShell `Invoke-RestMethod`、`curl` 或 `winget`（winget 不自動更新）。
3. 啟動方式：進入專案目錄後執行 `claude`，首次需設定色彩主題並登入帳號。
4. Claude 可存取你執行指令的目錄及其**所有子資料夾**。
5. 帳號類型：Pro、Max、Enterprise 或 API Key 皆可。

安裝指令速查 INSTALL COMMANDS

Mac / Linux / WSL

```
# 推薦（支援自動更新）
curl -fsSL https://claude.ai/install.sh | sh

# Homebrew（不自動更新）
brew install --cask claude-code
```

Windows

```
# PowerShell
Invoke-RestMethod -Uri https://claude.ai/install.ps1 | Invoke-Expression

# winget（不自動更新）
winget install Anthropic.ClaudeCode
```

各平台安裝方式 PLATFORM SETUP

平台	步驟	備註
Terminal	進入專案目錄 → 執行 <code>claude</code>	功能最完整
VS Code	Extensions → "Claude Code" (Anthropic 藍勾) → 安裝	Cmd+Shift+P 開啟
JetBrains	JetBrains Marketplace → 安裝後重啟	側邊欄出現 Claude logo
Claude Desktop	安裝後登入 → 頂部切換 "Code" 分頁	適合背景執行
Web	claude.ai/code	可在瀏覽器直接執行任務，不限 GitHub；支援桌面瀏覽器與 iOS app

關鍵詞彙 VOCABULARY

英文	中文	說明
Auto Update	自動更新	安裝後自動取得最新版本 (curl 安裝支援)
Project Directory	專案目錄	你要讓 Claude 操作的資料夾
API Key	API 金鑰	無帳號時可用此方式驗證身份

● 實作練習 TRY IT NOW

- 1 依照你的作業系統，用本節的指令安裝 Claude Code
- 2 執行 `claude --version` 確認安裝成功
- 3 選擇你最常用的平台 (Terminal / VS Code / Desktop)，完成對應的設定步驟
- 4 進入一個真實專案目錄，執行 `claude`，確認它能讀到你的檔案

我的筆記 MY NOTES

How Claude Code Works

Claude Code 如何運作

本節概念：Agentic Loop · Tools · Compaction · Permission Modes

► [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

1. **Agentic Loop（代理循環）**：Prompt → 收集情境 → 呼叫模型 → 執行行動 → 驗證結果 → 未達標則循環。
2. **Tools（工具）** 是 Agent 的核心骨架，讓 Claude 能「做事」而非只輸出文字。
3. 情境視窗接近上限時，Claude 會自動 **compact（壓縮）**，可能遺失部分細節。
4. 三種許可模式：預設（每次詢問）、Auto Accept（自動接受編輯）、Plan Mode（唯讀規劃）。
5. Claude Code 使用**語意搜尋（Semantic Search）** 判斷何時呼叫哪個工具。

AGENTIC LOOP 代理循環

```
graph TD
    A[使用者輸入 Prompt] --> B[收集所需情境 Context]
    B --> C[呼叫 LLM 回傳文字或工具呼叫]
    C --> D[執行行動 編輯檔案、執行指令...]
    D --> E[驗證結果]
    E --> F{達標?}
    F -- Yes --> G[等待下一個 Prompt]
    F -- No --> H[回到循環頂端]
```

三種許可模式 PERMISSION MODES

模式	說明	切換
Default 預設	每次操作前詢問許可	預設狀態
Auto Accept	自動接受檔案編輯；執行指令仍需確認	Shift + Tab
Plan Mode	僅使用唯讀工具，先規劃再執行	Shift + Tab（再切）

關鍵詞彙 VOCABULARY

英文	中文	說明
Agentic Loop	代理循環	Claude 自主執行任務的反覆循環機制
Tool Call	工具呼叫	Claude 決定使用特定工具的行動
Compaction	壓縮	情境視窗接近上限時，自動摘要舊內容釋放空間
Semantic Search	語意搜尋	根據語義判斷何時呼叫哪個工具
Verification	驗證	Claude 確認執行結果是否達到 prompt 目標

● 實作練習 TRY IT NOW

- 1 啟動 Claude Code，連按 **Shift + Tab** 幾次，觀察三種模式如何切換
- 2 在 Default 模式下，請 Claude 修改某個檔案——注意它如何詢問你的許可
- 3 切換到 Auto Accept 模式，執行同樣的任務，比較流程有何不同
- 4 觀察工具呼叫列表，確認你能看出 Agentic Loop 的每一個步驟

我的筆記 MY NOTES

Your First Claude Code Prompt

第一個 Prompt

本節概念：Prompt Writing · Plan Mode · Auto Accept

▶ [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

1. 用**自然語言**與 Claude Code 對話，如同對人類說話。
2. **Shift + Tab** 切換模式：Auto Accept ↔ 手動確認 ↔ Plan Mode。
3. **Plan Mode**：用唯讀工具分析程式碼庫 → 回傳詳細計畫 → 確認後才執行。
4. Prompt 原則：**越具體越好**，說清楚目標、現有限制、預期結果。
5. Plan Mode 的計畫可修改後再 Approve，是最佳的「開始執行前修正」節點。

PROMPT 寫作技巧

做 	不做 
說明「我需要達成什麼」	只說「幫我改這個」
描述現有系統的限制與偏好	不說明使用的框架或風格
指定預期結果的樣子	留下模糊的成功標準
一次聚焦一個目標	把太多功能塞進一個 prompt

PLAN MODE 使用流程

Shift + Tab → 進入 Plan Mode
輸入詳細 Prompt（說清楚目標與限制）
→ Claude 分析程式碼庫，返回計畫
審視計畫 → 修改 → Approve → Claude 開始執行

好的 PROMPT 範例

"My app needs a dark mode implemented across the entire app. Create a toggle switch on the header that allows user to toggle between light and dark mode. Find a good contrast color that works based on my existing light theme."

包含了：目標（dark mode）、位置（header toggle）、限制（配合現有 light theme）

關鍵詞彙 VOCABULARY

英文	中文	說明
Plan Mode	規劃模式	唯讀分析，先給計畫再執行
Auto Accept	自動接受	自動核准所有檔案編輯
Approve	核准	確認 Plan Mode 的計畫並開始執行
Course Correct	修正方向	執行中途調整 Claude 的做法

● 實作練習 TRY IT NOW

- 1 想一個你想在專案裡實作的小功能（例如：加一個按鈕、新增 API endpoint）
- 2 先用一句模糊的話下 prompt（例如：「加個 dark mode」），觀察 Claude 的回應
- 3 按照本節技巧，重新寫一個具體 prompt：包含目標、位置、限制、預期樣子
- 4 進入 Plan Mode，貼入具體 prompt，比較計畫的詳細度有何差異

我的筆記 MY NOTES

The CLAUDE.md File

CLAUDE.md 持久記憶檔案

本節概念：Persistent Memory · Hierarchy · /init

▶ [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

1. **CLAUDE.md = Claude Code 的持久記憶**，每次 session 啟動時自動讀取。
2. 沒有 CLAUDE.md 時，Claude 每次都要從頭理解程式碼庫，容易做出假設與錯誤。
3. `/init` 指令：讓 Claude 根據你的程式碼庫自動生成初始 CLAUDE.md。
4. 層級結構：**專案級**（根目錄，團隊共用）→ **使用者級**（個人設定，跨所有專案）。
5. 最佳實踐：先不加，觀察哪些地方需要反覆糾正，再精簡加入。

CLAUDE.MD 層級 HIERARCHY

<code>~/ .claude/CLAUDE.md</code>	← 使用者級（個人偏好，跨所有專案） 只有你看得到
<code>/your-project/CLAUDE.md</code>	← 專案級（根目錄，check in 版本控制） 整個團隊共用

CLAUDE.MD 建議填入內容

```
## Tech Stack 技術堆疊
- Next.js 15 (App Router) / Tailwind CSS / Drizzle ORM

## Commands 常用指令
- dev: npm run dev | test: npm test | lint: npm run lint

## Code Style 程式碼風格
- 2 space indentation | Named exports only

## Rules 規則
- Use server actions instead of API routes
```

最佳實踐 BEST PRACTICE

先不加 CLAUDE.md，觀察哪些地方需要反覆糾正，再把那些規則加進去。保持檔案精簡，只存放真正必要的資訊。

三個使用技巧

1. 糾正 Claude 的行為後，告訴他：「把這個規則存進 CLAUDE.md」
2. 用 @檔案路徑 引用文件，讓 Claude 直接參考
3. 從 /init 自動生成起點，再根據需要精簡

關鍵詞彙 VOCABULARY

英文	中文	說明
CLAUDE.md	—	Claude Code 的持久記憶設定檔（Markdown 格式）
Persistent Memory	持久記憶	跨 session 保留的知識與設定
/init	—	讓 Claude 自動生成 CLAUDE.md 的指令
Project-level	專案級	放在根目錄，整個團隊共用
User-level	使用者級	放在個人設定資料夾，只有自己用

● 實作練習 TRY IT NOW

- 1 在你的專案根目錄，執行 /init，讓 Claude 自動生成初始 CLAUDE.md
- 2 檢視生成的內容，確認技術堆疊、指令、風格描述是否正確，手動補齊遺漏的部分
- 3 加入一條你自己的規則（例如縮排、命名習慣、禁止某種寫法）
- 4 關掉 session，重新開啟，輸入同樣的任務——確認 Claude 已直接套用你的設定

我的筆記 MY NOTES

Explore → Plan → Code → Commit

核心工作流程

本節概念：Workflow · Plan Mode · Sub Agent · Commit

▶ [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

1. **核心工作流程**：Explore → Plan → Code → Commit，不要直接跳到 Code。
2. 多數人直接下指令產生程式碼，導致大量事後修正；**計畫確認前才是最佳修正節點**。
3. Plan Mode 可同時完成 Explore + Plan，是進入前兩步最快的方式。
4. 為 Claude 提供**可驗證的成功標準**（測試套件、Claude in Chrome），讓他自行確認完成。
5. Commit 前用 sub agent 做 code review，再讓 Claude 生成 commit message。

工作流程 WORKFLOW



備注：名稱差異

本講義依課程影片使用「**Code**」；Anthropic 官方文件（code.claude.com）將同一步驟標示為「**Implement**」。兩者指相同流程，「Implement」語意較廣，泛指所有實作工作——不限於程式設計，修改設定、資料遷移、撰寫文件等皆屬之。

Step 1 & 2 — Explore + Plan（用 Plan Mode）

Shift + Tab → 進入 Plan Mode → 輸入目標 Prompt
→ Claude 讀取檔案、搜尋文件、提問澄清
→ 回傳詳細計畫 → 審視 → 修改 → Approve

Step 3 — Code

讓 Claude 依計畫逐步執行；隨時 course-correct；把反覆修正的規則存進 CLAUDE.md。

Step 4 — Commit

- 1. 用 sub agent 做 code review
- 2. 讓 Claude 生成 commit message
- 3. Push → 開始下一個功能

讓 CLAUDE 自我驗證的工具

工具	用途
測試套件 (Test Suite)	Claude 跑測試確認功能正確
Claude in Chrome 擴充	Claude 控制瀏覽器實際測試 UI
Lint / Type Check	確保程式碼品質

關鍵詞彙 VOCABULARY

英文	中文	說明
Explore	探索	了解程式碼庫現狀，收集情境
Course Correct	修正方向	執行過程中調整 Claude 的做法
Sub Agent	子代理程式	獨立執行特定任務的 Claude 實例
Commit Message	提交訊息	Git commit 的描述文字

● 實作練習 TRY IT NOW

- 1 選一個你想實作的小功能，進入 Plan Mode，輸入完整 prompt
- 2 收到計畫後，找出 1-2 個你想調整的地方，要求 Claude 修改計畫
- 3 確認計畫符合預期後點 Approve，讓 Claude 開始執行
- 4 功能完成後，請 Claude 用 sub agent 做 code review，再生成 commit message 並提交

我的筆記 MY NOTES

Context Management

情境視窗管理

本節概念：/compact · /clear · /context · Sub Agent · Token

▶ [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

1. **Context Window = Claude 的工作記憶**，所有輸入、工具執行結果、對話都佔空間。
2. `/compact`：壓縮對話並保留摘要，繼續當前功能開發時使用。
3. `/clear`：完全清除，開始全新任務或新功能時使用。
4. `/context`：查看目前情境使用狀況、佔用分佈與視覺化圖表。
5. 減少消耗的技巧：提示詞要具體、關掉不用的 MCP server、善用 sub agent。

指令速查 COMMANDS

`/compact`

壓縮對話，保留摘要
繼續同一功能用

`/clear`

完全清除
開始新機能用

`/context`

查看 context
使用狀況

COMPACT VS CLEAR 決策樹

還在做同一個功能？

Yes → /compact（壓縮，保留摘要繼續工作）

No → /clear（清除，全新開始避免舊對話影響）

減少 CONTEXT 消耗的 5 個技巧

1. **提示詞要具體**：模糊 prompt 迫使 Claude 探索更多檔案
2. **停用不需要的 MCP server**：每個 server 的工具定義都佔 context
3. **用 sub agent 委派任務**：獨立 context，只回傳摘要給主 agent
4. **把常用規則放進 CLAUDE.md**：不需要 Claude 每次重新探索
5. **定期 /compact**：大 session 中途就壓縮，避免細節遺失

關鍵詞彙 VOCABULARY

英文	中文	說明
Context Window	情境視窗	Claude 的工作記憶容量
Compaction	壓縮	自動或手動摘要舊內容，釋放空間
Sub Agent	子代理程式	有獨立 context 的 Claude 實例，只回傳結果
Token	—	衡量 context 佔用量的單位

● 實作練習 TRY IT NOW

- 1 開啟一段正在進行的 session，執行 `/context`，看看哪些類別佔用最多空間
- 2 繼續做一段開發工作，再次執行 `/context`，觀察數字的變化
- 3 在功能還沒做完但 context 開始偏高時，執行 `/compact`，確認摘要是否保留了關鍵資訊
- 4 下次開始新功能前，養成先執行 `/clear` 的習慣，感受乾淨 context 帶來的差異

我的筆記 MY NOTES

MCP in Claude Code

整合外部工具

本節概念：Model Context Protocol · HTTP Server · stdio · Scope

▶ [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

1. **MCP (Model Context Protocol)** = 讓 Claude Code 連接外部工具與資料來源的開放標準。
2. 兩種 server 類型：**HTTP** (遠端服務，由服務商代管)、**stdio** (本機程式)。
3. 三種 scope：local (本機專案)、user (個人跨專案)、project (mcp.json，團隊共用)。
4. MCP server 的工具定義佔用 **context window**，不用的要用 `/mcp` 停用。
5. 工具佔用超過 10% context 時，Claude 自動切換 **tool search mode** (效果較差)。

三種 SCOPE (作用範圍)

Local 本機專案

只有你，只在這個專案有效

User 使用者

只有你，跨所有專案都可用

Project 專案級

整個團隊共用，透過
`.mcp.json` check in

常用操作

```
# 新增 MCP server
claude mcp add [server-name]

# 在 session 內管理 (查看狀態、停用)
/mcp
```

MCP VS CLI 選擇指南

情境	建議做法	原因
GitHub 操作	用 <code>gh</code> CLI	更節省 context
AWS 操作	用 <code>aws</code> CLI	更節省 context
Notion / Linear（無 CLI）	用 MCP server	唯一選項
團隊統一設定	project scope + <code>.mcp.json</code>	自動同步給所有人

關鍵詞彙 VOCABULARY

英文	中文	說明
MCP	Model Context Protocol	連接外部工具的開放標準
HTTP Server	HTTP 伺服器	遠端服務提供的 MCP server
stdio Server	標準 I/O 伺服器	在本機執行的 MCP server
Tool Search Mode	工具搜尋模式	工具太多時的自動備用模式，效果較差
<code>.mcp.json</code>	—	存放 project scope MCP 設定的檔案

● 實作練習 TRY IT NOW

- 1 在 session 中執行 `/mcp`，確認目前有哪些 server 連接，停用你現在不需要的
- 2 選一個與你專案相關的 MCP server 安裝（推薦：Context7 取得最新套件文件）
- 3 安裝後，要求 Claude 用這個工具完成一個真實任務，觀察它如何呼叫外部資料
- 4 如果你的團隊共用同一個 repo，建立 `.mcp.json` 並把 server 設定 check in

我的筆記 MY NOTES

Hooks in Claude Code

確定性自動化

本節概念：Deterministic · Hook Events · Exit Code · settings.json

► [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

1. **Hooks = 確定性執行的自動化**，不像 CLAUDE.md 的「建議」，Hooks 每次都一定執行。
2. 常見用途：自動格式化（prettier / gofmt）、記錄指令、封鎖危險操作、任務完成通知。
3. 設定在 **settings.json**，可 check in 版本控制讓整個團隊共用相同規則。
4. Exit code 規則：**0** = 繼續執行，**2** = 阻擋並把 stderr 回傳給 Claude。
5. 核心原則：某件事必須**每次都發生** → 放進 Hook，不要放進 prompt 或 CLAUDE.md。

「如果某件事必須每次都發生，不要放進 prompt，放進 Hook。」

5 種 HOOK 事件

事件	觸發時機	常見用途
UserPromptSubmit	你送出 prompt 時（Claude 處理前）	前置驗證、記錄
PreToolUse	工具呼叫前	封鎖危險操作
PostToolUse	工具呼叫後	自動格式化、記錄
Notification	Claude 發送通知時	轉發到 Slack / 系統通知
Stop	Claude 完成回應時	任務完成通知

EXIT CODE 規則

Exit 0

繼續執行（proceed）
工具呼叫正常進行

Exit 2

阻擋（block）
stderr 回傳給 Claude，Claude 知道原因後調整

最常用：自動格式化 POSTTOOLUSE HOOK

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|MultiEdit",
      "command": "bash -c 'ext=${TOOL_INPUT_FILE##*.}';
        case $ext in
          ts|tsx) prettier --write $TOOL_INPUT_FILE ;;
          go)      gofmt -w $TOOL_INPUT_FILE ;;
          py)      ruff format $TOOL_INPUT_FILE ;;
          esac'"
    ]}
}
```

CLAUDE.MD VS HOOKS 比較

	CLAUDE.md	Hooks
執行方式	建議（Claude 決定是否遵循）	確定性（每次都執行）
適合場景	偏好、風格、技術堆疊說明	格式化、封鎖、通知
設定位置	根目錄 Markdown 檔	settings.json
可程式化	否	是（shell script）

關鍵詞彙 VOCABULARY

英文	中文	說明
Hook	鉤子	在特定事件觸發時確定性執行的命令
Deterministic	確定性的	每次都保證執行，無例外
Matcher	匹配器	指定 Hook 作用於哪個工具
Exit Code	退出碼	0 = 繼續，2 = 阻擋
PreToolUse	工具呼叫前	可用來驗證或阻擋某些操作
PostToolUse	工具呼叫後	常用於格式化、記錄

● 實作練習 TRY IT NOW

- 1 在 settings.json 加入一個 PostToolUse hook，讓每次 Claude 編輯檔案後自動執行格式化（參考本節的範例 JSON）
- 2 請 Claude 修改一個檔案，確認 hook 有自動觸發、格式化有實際執行
- 3 加入一個 PreToolUse hook，封鎖一個你覺得危險的操作（例如：包含 `rm -rf` 的指令）
- 4 盤點你的工作流程中有哪些「一定要做」的步驟，把它們都移進 hook——讓規則不再依賴你記得說

! 我的筆記 MY NOTES
